Java Backend Architecture

Interview Series

Chapter 12.6
Character Encoding

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

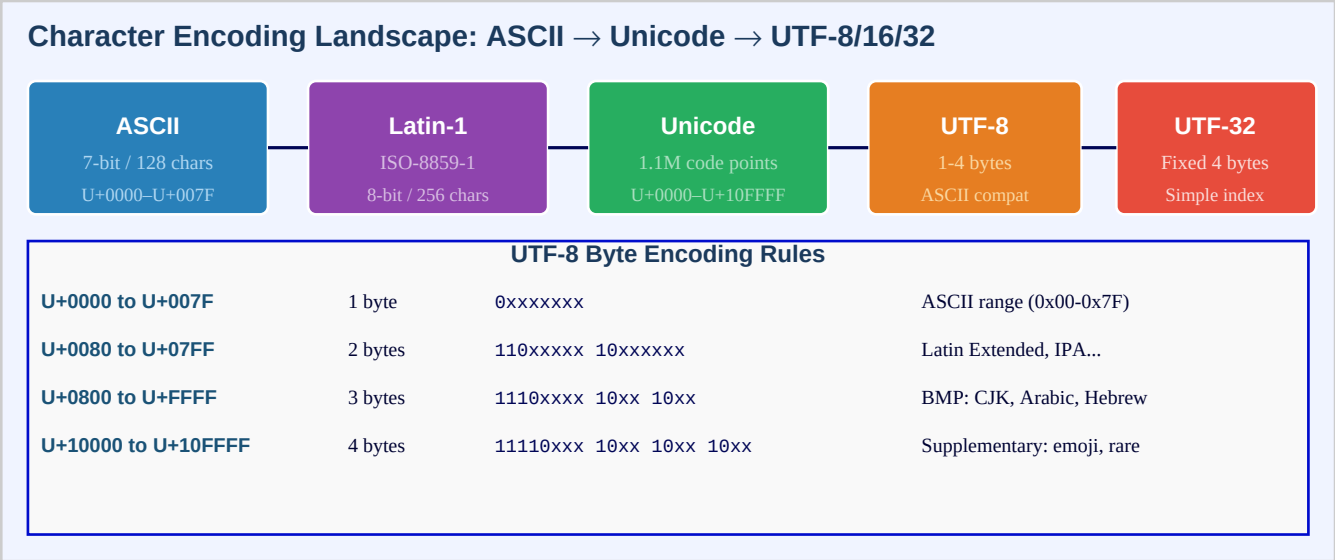
Chapter 12.6 – Character Encoding

Introduction

Character encoding is the foundation of correct text handling in any backend system. At its core, encoding defines how abstract Unicode code points — the canonical numeric identifiers for every character in human writing — are serialized as sequences of bytes. ASCII, the original 7-bit standard, covers 128 characters: English letters, digits, and control characters. Latin-1 (ISO-8859-1) extends this to 256 characters by using the full 8-bit byte, covering Western European languages. Unicode, the modern universal standard, assigns code points to over 140,000 characters across all writing systems plus emoji. The challenge for engineers is that Unicode itself is just a numbering scheme — the physical bytes on disk or in a network packet are determined by the encoding format: UTF-8, UTF-16, or UTF-32.

UTF-8 has become the dominant encoding on the web and in modern systems because it is backward compatible with ASCII (the first 128 code points use identical byte values), variable-width (1–4 bytes per code point), and self-synchronizing. Java internally represents String objects using UTF-16, which means that characters outside the Basic Multilingual Plane — such as emoji and rare CJK extensions — require two char values (a surrogate pair), making String.length() return a value larger than the perceived character count. Engineers must account for this in string length validation, truncation, and database column sizing. Always specify encoding explicitly — never rely on the platform default, which differs between development machines and production servers.

Encoding Landscape Overview



UTF-16 and Surrogate Pairs

UTF-16: BMP (2 bytes) vs Supplementary Planes (Surrogate Pairs)

Basic Multilingual Plane (BMP)

Code points: U+0000 to U+FFFF

Encoded as: exactly 2 bytes

Example: U+0041 (A) = 0x0041 in UTF-16LE

Supplementary Planes (> U+FFFF)

Code points: U+10000 to U+10FFFF

Encoded as: surrogate PAIR (4 bytes)

High: 0xD800-0xDBFF Low: 0xDC00-0xDFFF

Java String Internal Representation (UTF-16)

String.length() returns number of char (UTF-16 code units), NOT Unicode code points!

```
"hello".length() = 5  (correct)
"emoji".codePointCount(0, s.length()) = 1  (emoji = 2 chars, 1 code point)
```

Use s.codePoints().count() for true character count

BOM: U+FEFF at stream start identifies UTF-16 byte order (BE vs LE)

Charset in Java I/O, HTTP and Databases

Character Encoding in Java I/O, HTTP and Database

Java I/O

```
new InputStreamReader(
  stream, StandardCharsets.UTF_8)
Files.writeString(path, str, UTF_8)
str.getBytes(StandardCharsets.UTF_8)
```

HTTP Headers

Content-Type:

```
  text/html; charset=UTF-8
Accept-Charset: utf-8
```

Spring: produces=MediaType.UTF_8

Database

MySQL utf8mb4 (emoji!)

MySQL utf8 = 3-byte only

PostgreSQL: UTF8 default

collation: utf8mb4_unicode_ci

Base64 — Encoding Binary as ASCII Text

Purpose: transport binary (images, keys) in text-only contexts (JSON, HTTP headers, email)

Each 3 bytes of input -> 4 ASCII chars (A-Z, a-z, 0-9, +, /); 33% overhead

```
Base64.getEncoder().encodeToString(bytes)  /  Base64.getDecoder().decode(str)
```

URL-safe variant: Base64.getUrlEncoder() replaces + with - and / with _

Base64 is NOT encryption — it is encoding; do not use for security purposes

Key Definitions

Code point	An integer value in the Unicode standard uniquely identifying a character: U+0041 = Latin capital A; U+1F600 = grinning face emoji
ASCII	7-bit encoding; 128 characters (0x00-0x7F); covers English alphabet, digits, control codes; foundational subset of all modern encodings
Latin-1 / ISO-8859-1	8-bit encoding; 256 characters; extends ASCII with Western European accented characters (U+00A0–U+00FF); NOT the same as UTF-8 for bytes > 127
Unicode	Universal character standard maintained by the Unicode Consortium; over 140,000 code points across 17 planes (U+0000 to U+10FFFF)
BMP	Basic Multilingual Plane (Plane 0): U+0000–U+FFFF; contains most common characters; encoded in 2 bytes by UTF-16
Supplementary plane	Unicode planes 1–16 (U+10000–U+10FFFF); includes emoji, historic scripts, rare CJK; requires surrogate pairs in UTF-16 and 4 bytes in UTF-8

UTF-8	Variable-width Unicode encoding; 1 byte for ASCII, up to 4 bytes for supplementary; backward compatible with ASCII; most common on the web
UTF-16	Variable-width encoding; 2 bytes for BMP characters; 4 bytes (surrogate pair) for supplementary; Java String internal format; BOM required for file interchange
UTF-32	Fixed-width 4-byte encoding; every code point uses exactly 4 bytes; simple random access by index; wastes memory; rare in practice
BOM	Byte Order Mark (U+FEFF); optional in UTF-8 (0xEF 0xBB 0xBF); required in UTF-16 to indicate byte order (big-endian vs little-endian)
Surrogate pair	Pair of UTF-16 code units (high U+D800-U+DBFF and low U+DC00-U+DFFF) that together encode a supplementary Unicode code point
StandardCharsets	Java class (java.nio.charset.StandardCharsets) with constants UTF_8, UTF_16, ISO_8859_1, US_ASCII; always use instead of string literals
utf8mb4	MySQL charset that supports the full 4-byte UTF-8 (all Unicode including emoji); the older MySQL "utf8" only supports 3-byte sequences (up to U+FFFF)
Base64	Binary-to-text encoding scheme; converts 3 bytes to 4 printable ASCII characters; used to embed binary data in JSON, HTTP headers, or email
charset parameter	In HTTP Content-Type header: "Content-Type: text/html; charset=UTF-8" — tells the receiver how to decode the body bytes into characters

Encoding Comparison Table

Encoding	Bytes per char	Range	BOM	ASCII compat	Best used for
ASCII	1 (fixed)	U+0000–U+007F	No	Yes	Legacy English-only systems
ISO-8859-1	1 (fixed)	U+0000–U+00FF	No	Yes	Western European legacy text
UTF-8	1–4 (var)	U+0000–U+10FFFF	Optional	Yes	Web, APIs, files — default choice
UTF-16	2 or 4 (var)	U+0000–U+10FFFF	Required	No	Java internals, Windows APIs
UTF-32	4 (fixed)	U+0000–U+10FFFF	Optional	No	Internal processing needing random access
Base64	4 per 3 src	Printable ASCII	No	N/A	Binary data in text contexts

Code Examples

1. String.getBytes and new String with explicit charset

```
import java.nio.charset.StandardCharsets; import java.util.Arrays; String text = "Hello, World! éàü"; // accented chars // WRONG: relies on platform default encoding (may differ dev vs server) byte[] wrongBytes = text.getBytes(); // platform default — unpredictable! // CORRECT: always explicit byte[] utf8Bytes = text.getBytes(StandardCharsets.UTF_8); byte[] latin1Bytes = text.getBytes(StandardCharsets.ISO_8859_1); System.out.println("UTF-8 length: " + utf8Bytes.length); // > string length (multi-byte) System.out.println("Latin-1 length: " + latin1Bytes.length); // == string.length() // Decode bytes back to String String decoded = new String(utf8Bytes, StandardCharsets.UTF_8); System.out.println(text.equals(decoded)); // true // WRONG decode — mixing encodings corrupts data String corrupted = new String(utf8Bytes, StandardCharsets.ISO_8859_1); System.out.println(corrupted); // garbled characters! // Code
```

```
point vs char count for emoji String emoji = "■■"; // U+1F600 grinning face (surrogate pair in Java) System.out.println(emoji.length()); // 2 (two char values) System.out.println(emoji.codePointCount(0, emoji.length())); // 1 (one Unicode character) System.out.println(emoji.chars().count()); // 2 (chars stream, not code points) System.out.println(emoji.codePoints().count()); // 1 (code points stream — correct!)
```

2. InputStreamReader and file I/O with explicit charset

```
import java.io.*; import java.nio.charset.StandardCharsets; import java.nio.file.*; // WRONG: InputStreamReader uses platform default charset BufferedReader badReader = new BufferedReader( new InputStreamReader(System.in)); // platform default! // CORRECT: always specify charset BufferedReader goodReader = new BufferedReader( new InputStreamReader(System.in, StandardCharsets.UTF_8)); // File I/O with charset (Java 11+) Path file = Path.of("/tmp/data.txt"); String content = Files.readString(file, StandardCharsets.UTF_8); // read Files.writeString(file, "Hello ■■", StandardCharsets.UTF_8); // write CJK chars // Older style with explicit charset try (BufferedWriter w = Files.newBufferedWriter(file, StandardCharsets.UTF_8); BufferedReader r = Files.newBufferedReader(file, StandardCharsets.UTF_8)) { w.write("UTF-8 content: éà"); w.newLine(); String line = r.readLine(); } // Reading HTTP response body HttpClient client = HttpClient.newHttpClient(); HttpRequest request = HttpRequest.newBuilder() .uri(URI.create("https://example.com/api")) .build(); HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString(StandardCharsets.UTF_8)); // HttpResponse.BodyHandlers.ofString() uses charset from Content-Type header if present
```

3. UTF-16 surrogate pairs and code point iteration

```
// Java String internals: UTF-16, surrogates for supplementary chars // BMP character (U+4E16 = Chinese "world") String cjk = "■"; System.out.println(cjk.length()); // 1 char System.out.println(Integer.toHexString(cjk.codePointAt(0))); // 4e16 // Supplementary character: U+1F600 emoji (beyond BMP) // In Java source: use \uD83D\uDE00 (surrogate pair) or \u{1F600} (Java 13+ text block escape) String face = "■■"; System.out.println(face.length()); // 2 (two char values!) System.out.println(Character.isSurrogate(face.charAt(0))); // true (high surrogate) System.out.println(Character.isSurrogate(face.charAt(1))); // true (low surrogate) System.out.println(face.codePointAt(0)); // 128512 = 0x1F600 System.out.println(face.codePointCount(0, face.length())); // 1 // SAFE iteration over code points (handles surrogates) "Hello ■■ World".codePoints().forEach(cp -> { System.out.printf("U+%04X %s%n", cp, new String(Character.toChars(cp))); }); // UNSAFE iteration (may split surrogate pairs) for (char c : "Hello ■■".toCharArray()) { System.out.println(c); // '■' and '■' printed separately — meaningless } // Correct string reversal (preserves surrogate pairs) String reversed = new StringBuilder("Hello ■■") .reverse().toString(); // StringBuilder.reverse() handles surrogates correctly
```

4. Charset in HTTP Content-Type and Spring MVC

```
import org.springframework.web.bind.annotation.*; import org.springframework.http.*; // Spring MVC: specify charset in produces @RestController @RequestMapping("/api") public class TextController { // Explicit charset in produces @GetMapping(value = "/message", produces = MediaType.APPLICATION_JSON_VALUE + ";charset=UTF-8") public ResponseEntity<String> getMessage() { return ResponseEntity.ok("Hello ■■! Emoji: ■■"); } } // application.properties — force UTF-8 for all responses // server.servlet.encoding.charset=UTF-8 // server.servlet.encoding.force=true // server.servlet.encoding.force-response=true // Spring Boot auto-config: CharacterEncodingFilter // Equivalent manual config: @Bean public FilterRegistrationBean<CharacterEncodingFilter> encodingFilter() { FilterRegistrationBean<CharacterEncodingFilter> reg = new FilterRegistrationBean<>(); CharacterEncodingFilter f = new CharacterEncodingFilter(); f.setEncoding("UTF-8");
```

```
f.setForceEncoding(true); reg.setFilter(f); reg.addUrlPatterns("/*"); return reg; } // Parsing charset from Content-Type header String contentType = "text/html; charset=ISO-8859-1"; MediaType mt = MediaType.parseMediaType(contentType); Charset charset = mt.getCharset(); // Charset.forName("ISO-8859-1") // HttpMessageConverter with charset StringHttpMessageConverter converter = new StringHttpMessageConverter(StandardCharsets.UTF_8);
```

5. MySQL utf8mb4 configuration and emoji support

```
-- MySQL: utf8 (3-byte) vs utf8mb4 (4-byte, full Unicode) -- The problem: MySQL's old "utf8" only stores up to U+FFFF (3 bytes) -- Emoji like U+1F600 require 4 bytes → INSERT fails silently or throws -- Fix: use utf8mb4 everywhere -- Database level CREATE DATABASE myapp CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci; -- Table level CREATE TABLE messages ( id BIGINT AUTO_INCREMENT PRIMARY KEY, content TEXT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci, author VARCHAR(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci ); -- Alter existing table to utf8mb4 ALTER TABLE messages CONVERT TO CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci; -- MySQL connection parameter (JDBC URL) spring.datasource.url=jdbc:mysql://localhost/myapp?characterEncoding=utf8mb4&useUnicode=true -- MySQL server config (my.cnf) -- [mysqld] -- character-set-server = utf8mb4 -- collation-server = utf8mb4_unicode_ci -- Verify SHOW VARIABLES LIKE 'character_set%'; -- character_set_client: utf8mb4 -- character_set_connection: utf8mb4 -- character_set_results: utf8mb4 -- collation comparison -- utf8mb4_general_ci: faster, less accurate (treats ä = a) -- utf8mb4_unicode_ci: accurate Unicode collation -- utf8mb4_bin: case-sensitive, byte comparison
```

6. Base64 encoding and decoding in Java

```
import java.util.Base64; import java.nio.charset.StandardCharsets; // Encode bytes to Base64 string byte[] originalBytes = "Hello, World!".getBytes(StandardCharsets.UTF_8); String encoded = Base64.getEncoder().encodeToString(originalBytes); System.out.println(encoded); // SGVsbG8sIFdvcmxkIQ== // Decode Base64 string back to bytes byte[] decodedBytes = Base64.getDecoder().decode(encoded); String decoded = new String(decodedBytes, StandardCharsets.UTF_8); System.out.println(decoded); // Hello, World! // URL-safe Base64 (replaces + with - and / with _) // Used in JWT, URL parameters, filenames String urlEncoded = Base64.getUrlEncoder().withoutPadding() .encodeToString(originalBytes); System.out.println(urlEncoded); // SGVsbG8sIFdvcmxkIQ (no trailing ==) byte[] urlDecoded = Base64.getUrlDecoder().decode(urlEncoded); // MIME Base64 (adds line breaks every 76 chars — for email) String mimeEncoded = Base64.getMimeEncoder().encodeToString(originalBytes); // Encoding an image for embedding in JSON byte[] imageBytes = Files.readAllBytes(Path.of("/tmp/icon.png")); String base64Image = Base64.getEncoder().encodeToString(imageBytes); String jsonPayload = "{"icon": "" + base64Image + ""}"; // JWT structure: header.payload.signature (all Base64url encoded) // Header: {"alg":"HS256","typ":"JWT"} String jwtHeader = Base64.getUrlEncoder().withoutPadding() .encodeToString("{"alg":"HS256","typ":"JWT"}" .getBytes(StandardCharsets.UTF_8)); System.out.println(jwtHeader); // eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 // Base64 is NOT encryption — never use it to "hide" sensitive data // It increases size by ~33%; use only when binary-in-text is needed
```

Interview Questions & Answers

Q1. What is the difference between a character set, a code point, and an encoding?

A character set (or repertoire) is the collection of abstract characters — for example, the Unicode character set includes every letter, digit, symbol, and emoji across all writing systems. A code point is the unique integer assigned to each character in that set: U+0041 is the code point for Latin capital A, U+1F600 for the grinning

face emoji. An encoding is the rule that maps code points to bytes: UTF-8 encodes U+0041 as one byte 0x41, while U+1F600 requires four bytes 0xF0 0x9F 0x98 0x80. You can think of it as: code points are abstract 'addresses'; encodings are the physical byte representation.

Q2. Why does Java's String.length() sometimes give a larger number than the visible character count?

Java String stores characters in UTF-16 internally. Characters in the Basic Multilingual Plane (U+0000–U+FFFF) are represented as a single char (2 bytes). Characters in supplementary planes — such as emoji (e.g., U+1F600) or rare CJK extensions — require two char values called a surrogate pair. String.length() returns the number of char values, not the number of Unicode code points. So the emoji string '\uD83D\uDE00' has length() == 2 but contains only one character. Use s.codePointCount(0, s.length()) or s.codePoints().count() for the actual character count. This matters for input validation, database VARCHAR sizing, and UI character limits.

Q3. What is UTF-8 and why is it the preferred encoding for web APIs?

UTF-8 is a variable-width encoding: ASCII characters (U+0000–U+007F) use exactly 1 byte (same byte value as ASCII — backward compatible). Characters U+0080–U+07FF use 2 bytes, U+0800–U+FFFF use 3 bytes, and supplementary planes use 4 bytes. Advantages: (1) ASCII compatibility — all ASCII text is valid UTF-8 with no conversion. (2) No byte order issues — no BOM required (unlike UTF-16). (3) Self-synchronizing — you can find character boundaries by scanning from any byte. (4) Compact for ASCII-heavy content (English, JSON keys, XML). Disadvantages: random access by character index is O(n), not O(1). For APIs: always specify charset=utf-8 in Content-Type.

Q4. Explain UTF-16 surrogate pairs and when they matter in Java.

The Unicode BMP covers U+0000–U+FFFF (65,536 code points), all representable in 2 bytes. To encode the 1,048,576 supplementary code points (U+10000–U+10FFFF), UTF-16 uses surrogate pairs: a high surrogate (U+D800–U+DBFF) followed by a low surrogate (U+DC00–U+DFFF), totalling 4 bytes. In Java, surrogates matter when: (1) Counting characters — use codePoints() stream, not chars(). (2) Truncating strings — substring() at a surrogate boundary produces invalid UTF-16. (3) Regular expressions — Pattern's \\X matches a full grapheme cluster including surrogates. (4) Database storage — MySQL's utf8 (3-byte max) rejects supplementary characters; use utf8mb4.

Q5. What is the difference between UTF-8 and UTF-16 BOM handling?

BOM (Byte Order Mark) is the code point U+FEFF placed at the start of a text stream. UTF-16 requires a BOM because the 2-byte encoding depends on byte order: UTF-16 BE (big-endian, 0xFE 0xFF) vs UTF-16 LE (little-endian, 0xFF 0xFE). Without a BOM, the receiver must assume UTF-16 BE (per spec) or negotiate out-of-band. UTF-8 BOM (0xEF 0xBB 0xBF) is optional and often causes problems: BOM-prefixed UTF-8 files break scripts that check for exact byte sequences at the start (e.g., shell shebang lines, CSV parsers, JSON parsers). Windows Notepad historically saved UTF-8 with BOM — avoid BOM in UTF-8 files for cross-platform compatibility.

Q6. Why must you always specify charset explicitly in Java I/O, and what is the risk of not doing so?

When you use new InputStreamReader(stream) without a charset, Java uses Charset.defaultCharset(), which is the JVM platform default — typically the OS locale's charset. On macOS/Linux this is usually UTF-8; on Windows it may be windows-1252. Consequence: code that works on a developer's Mac (default UTF-8) silently corrupts data on a Windows CI server (default windows-1252) for any characters above U+007F. Always use: new InputStreamReader(stream, StandardCharsets.UTF_8), Files.readString(path, UTF_8), and String.getBytes(StandardCharsets.UTF_8). The PMD rule 'AvoidUsingHardCodedIP' and SpotBugs DM_DEFAULT_ENCODING flag charset-less I/O calls.

Q7. What is MySQL's utf8mb4 and why is the plain utf8 charset insufficient?

MySQL's historical 'utf8' character set only supports up to 3-byte UTF-8 sequences, covering the BMP (U+0000–U+FFFF). This means it cannot store supplementary characters including emoji (U+1F600 and above), Mahjong tiles, musical symbols, and some CJK extensions. Attempting to insert a 4-byte character into a utf8 column either silently truncates the string or throws an error depending on SQL mode. utf8mb4 is MySQL's full 4-byte UTF-8 support, matching the actual UTF-8 standard. Migration: ALTER TABLE t CONVERT TO CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci; Also set the JDBC URL parameter characterEncoding=utf8mb4.

Q8. How do you handle character encoding correctly in a Spring REST API?

1. Set server.servlet.encoding.charset=UTF-8 and server.servlet.encoding.force=true in application.properties. 2. In @RequestMapping, use produces = 'application/json;charset=UTF-8'. 3. Spring Boot auto-configures CharacterEncodingFilter — verify it is not disabled. 4. Jackson ObjectMapper (used by default for JSON) writes UTF-8 by default. 5. For file uploads, read InputStream bytes with explicit charset. 6. In response headers: set Content-Type explicitly when returning text/plain or text/html. 7. Do not call response.getWriter() before setting charset — the charset must be set before the writer is obtained. HttpServletResponse.setCharacterEncoding('UTF-8') must precede getWriter().

Q9. What is Base64 and when should you use it?

Base64 is a binary-to-text encoding that converts arbitrary byte sequences to a printable ASCII subset (A-Z, a-z, 0-9, +, /). Every 3 input bytes produce 4 output characters (33% size overhead). Padding ('=') aligns to 4-character boundaries. Use cases: (1) Embedding binary data (images, certificates) in JSON, XML, or email (MIME). (2) Encoding binary values in HTTP headers (Authorization: Basic). (3) JWT — header and payload are Base64url-encoded. (4) URL-safe variant replaces + with - and / with _ for embedding in URLs without percent-encoding. Base64 is NOT encryption — it is trivially reversible by anyone. Never use it to 'hide' passwords or sensitive data.

Q10. How do you correctly count characters in a user-submitted string for a VARCHAR(255) database column?

A VARCHAR(255) in MySQL (utf8mb4) stores 255 characters, where each character can occupy 1-4 bytes. String.length() in Java counts UTF-16 code units, not characters. For BMP-only content (no emoji): length() == character count. For mixed content: use s.codePointCount(0, s.length()) to count Unicode code points. For database VARCHAR limit: the limit is in characters (code points), not bytes in MySQL, so codePointCount is the right measure. However, some databases limit by bytes: PostgreSQL VARCHAR(n) limits by character count but the total row size is limited to ~8kB. For UI input length validation: codePointCount. For HTTP header size limits: byte count after encoding.

Q11. What is the difference between character encoding and collation?

Character encoding defines how characters are stored as bytes (UTF-8, UTF-16, etc.). Collation defines the rules for comparing and sorting characters for a given encoding. Example: utf8mb4_unicode_ci (MySQL) — case-insensitive, accent-insensitive Unicode comparison where 'A' == 'a' and 'e' == 'é'. utf8mb4_bin — binary comparison, case-sensitive, accent-sensitive (each byte compared directly). utf8mb4_unicode_ci uses the Unicode Collation Algorithm (UCA) for culturally correct sorting: Swedish treats ä after z; German sorts ä as ae. For PostgreSQL: use the 'C' locale for byte sorting or 'en_US.UTF-8' for natural English sorting. Collation affects ORDER BY, WHERE with string comparison, and unique constraints.

Q12. How does the Content-Type charset parameter affect Spring MVC request parsing?

When a client sends a POST request with Content-Type: application/x-www-form-urlencoded; charset=ISO-8859-1, Spring's CharacterEncodingFilter must be configured to force UTF-8 or the filter must be placed before other filters to set encoding before parsing. For JSON (application/json): Jackson reads the raw

bytes and detects encoding from the BOM or defaults to UTF-8 per RFC 8259. For text/plain: Spring uses the charset from Content-Type header, falling back to ISO-8859-1 if absent (HTTP/1.1 default for text/* types). For multipart/form-data file uploads: each part can have its own charset header. Best practice: always send Content-Type: application/json; charset=utf-8 from clients.

Q13. Explain the difference between new String(bytes) and new String(bytes, charset).

new String(bytes) without charset uses the JVM's default charset (Charset.defaultCharset()), which varies by OS and JVM startup configuration. On a UTF-8 Linux server this behaves correctly; on a windows-1252 Windows machine it mangles characters above U+007F. new String(bytes, StandardCharsets.UTF_8) always uses UTF-8 regardless of environment. This is one of the most common sources of encoding bugs in Java code: data is stored correctly in a database but displayed incorrectly because decoding used the wrong charset. Tools: SpotBugs rule DM_DEFAULT_ENCODING catches calls without explicit charset. The fix is always to pass StandardCharsets.UTF_8 explicitly.

Q14. How do you handle encoding in a Properties file loaded by Spring?

Java .properties files are historically encoded in ISO-8859-1 with Unicode escape sequences for non-Latin characters: \u4e2d\u6587 for Chinese text. Spring Boot supports UTF-8 .properties files natively since Spring Boot 2.x by default. For message source / i18n bundles: spring.messages.encoding=UTF-8 in application.properties. ResourceBundleMessageSource has setDefaultEncoding("UTF-8"). When using @PropertySource: @PropertySource(value = "classpath:config.properties", encoding = "UTF-8"). YAML files (application.yml) are always UTF-8. Recommendation: use UTF-8 for all property files and avoid Unicode escape sequences.

Q15. What is the Unicode Normalization Form and when does it matter in Java?

Unicode allows the same visual character to be represented in multiple byte sequences. Example: 'é' can be U+00E9 (precomposed, one code point) or U+0065 U+0301 (decomposed, 'e' + combining accent, two code points). Both look identical but are different byte sequences and compare unequal with String.equals(). Normalization Forms: NFC (Canonical Decomposition then Composition — most compact precomposed); NFD (Canonical Decomposition — expanded combining characters); NFKC/NFKD (Compatibility normalization). Java: java.text.Normalizer.normalize(str, Form.NFC). When it matters: username comparison, search indexing, password hashing, URL paths. Normalize to NFC before storing or comparing user-provided text.

Q16. How do you debug character encoding issues in a Java backend?

Step 1: Identify the corruption point — is it in the database, the API response, or the Java processing layer? Step 2: Log byte arrays at each stage: Arrays.toString(str.getBytes(UTF_8)) to see the raw bytes. Step 3: Check database charset: SHOW CREATE TABLE t; — verify utf8mb4. Check JDBC URL: characterEncoding=utf8mb4. Step 4: Check HTTP response headers — use browser dev tools to verify Content-Type: application/json; charset=utf-8. Step 5: Check properties files for non-UTF-8 encoding or missing escape sequences. Step 6: Enable SQL logging to see the actual bytes sent to the database. Step 7: Use curl -v to inspect raw HTTP headers. Common root cause: server JVM timezone default is windows-1252, or MySQL connection encoding not set to utf8mb4.

Q17. What Java APIs help with character set detection and conversion?

java.nio.charset.Charset: Charset.forName("UTF-8"), Charset.availableCharsets(), StandardCharsets constants (UTF_8, UTF_16, ISO_8859_1, US_ASCII). CharsetDecoder: fine-grained decoding with CodingErrorAction.REPLACE or .REPORT to handle malformed bytes. CharsetEncoder for the reverse. ICU4J library (com.ibm.icu): CharsetDetector class that guesses encoding from byte samples — useful for legacy files with unknown encoding. Apache Tika: detects charset and content type from file bytes. HttpClient BodyHandlers.ofString() uses charset from Content-Type header automatically. URL.openStream() with

InputStreamReader — always add charset. Avoid guessing; when charset is unknown, ask the data provider.

Q18. How should you handle encoding when writing CSV files for Excel compatibility?

Excel on Windows expects CSV files in the system codepage (typically windows-1252 on Western European systems) or UTF-8 with a BOM. Without the BOM, Excel treats UTF-8 as the system default codepage and mangles non-ASCII characters. Solution: write a UTF-8 BOM at the start of the file: outputStream.write(new byte[]{(byte)0xEF,(byte)0xBB,(byte)0xBF}); then write UTF-8 content. This is the one case where the UTF-8 BOM is intentionally included. Apache POI alternative: write .xlsx (OOXML) directly — no charset issue since OOXML is always UTF-8 internally. For CSV libraries: OpenCSV, Apache Commons CSV support BOM-prefixed UTF-8 output.

Q19. What is the difference between charset and encoding in HTTP?

In HTTP, 'charset' is the parameter in the Content-Type header specifying the character encoding of the body: Content-Type: text/html; charset=utf-8. Despite the name 'charset', the value is actually an encoding name (UTF-8, windows-1252, ISO-8859-1). RFC 2045 and RFC 7231 use 'charset' to mean encoding. For application/json: RFC 8259 mandates UTF-8 for JSON interoperability — the charset parameter is technically unnecessary but including it improves clarity. For text/plain: the default charset per HTTP/1.1 is ISO-8859-1 if not specified — a common source of bugs when sending UTF-8 text without the charset parameter.

Q20. How does Base64url differ from standard Base64 and where is it used?

Standard Base64 uses the characters A-Z, a-z, 0-9, +, / and padding =. The + and / characters have special meaning in URLs (+ = space in form encoding, / = path separator) and must be percent-encoded as %2B and %2F. Base64url replaces + with - and / with _ and optionally omits padding =, making the encoded string safe to embed directly in URLs and filenames without percent-encoding. Used in: JWT (header, payload, signature), OAuth state parameters, PKCE code verifier/challenge, file names derived from content hash. Java: Base64.getUrlEncoder() and Base64.getUrlDecoder(). Apache Commons Codec: Base64.encodeBase64URLSafeString().

Q21. What encoding issues arise with emoji in Java applications and how do you fix them?

Emoji are typically in Unicode supplementary planes (U+1F000+), requiring 4 bytes in UTF-8 and 2 chars (surrogate pair) in Java's UTF-16 String. Issues: (1) MySQL utf8 (3-byte) rejects emoji inserts — fix: migrate to utf8mb4. (2) String.length() double-counts emoji — fix: use codePointCount. (3) Truncation at a surrogate boundary produces invalid string — fix: truncate by code points using codePoints().limit(n).collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append).toString(). (4) Some regex . patterns don't match supplementary chars — fix: use Pattern.UNICODE_CHARACTER_CLASS. (5) Jackson defaults serialize emoji correctly in UTF-8 JSON. (6) Log4j/SLF4J: ensure log appender uses UTF-8 charset.